

使用 Unity 的 AR Foundation 建立 AR 遊戲

來源：<https://codelabs.developers.google.com/arcore-unity-ar-foundation?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將瞭解如何運用 Unity 的 AR 基礎架構使用 ARCore 打造簡單的駕駛遊戲。

目錄

1. [1. 總覽](#)
2. [2. 設定開發環境](#)
3. [3. 偵測現實世界中的飛機](#)
4. [4. 對偵測到的平面執行命中測試](#)
5. [5. 設定光線估算](#)
6. [6. 總結](#)

步驟 1 · 約 0 分鐘

1. 總覽

ARCore 是 Google 的架構，可在智慧型手機上打造擴增實境體驗。您可以使用 Unity 的 AR Foundation 建構跨平台 AR 應用程式。

建構項目

在本程式碼研究室中，您將使用 AR Foundation 建構簡單的遊戲。遊戲目標是使用手持裝置操控車輛，收集包裹。

不過，這在完全虛擬的世界中不會發生！您將混合實體原子和數位位元，建立可瞭解玩家周遭環境的遊戲，打造全新類型的玩家體驗。

完成本程式碼研究室後，您的遊戲將能夠：

- 偵測現實世界中的平面，並在平面上繪製球場。
- 從攝影機的視野投射光線，並偵測與平面相交的位置。
- 根據現實世界的光線條件做出反應，為遊戲增添真實感。



課程內容

- 如何設定使用 Unity AR Foundation 的專案。
- 如何使用 `ARPlaneManager` 訂閱新方案。
- 如何使用 `Raycast` 尋找與虛擬幾何的交集
- 如何使用 `ARLightEstimationData` 照亮場景。

軟硬體需求

- [支援 ARCore 的裝置](#)，透過 USB 傳輸線連接至開發機器。
 - [Google Play 服務 - AR 適用](#) 1.23 以上版本。
 - 安裝 [Unity Hub](#) 或 [Unity 2020 LTS](#)。
-

步驟 2 · 約 0 分鐘

2. 設定開發環境

在這個步驟中，您將準備好環境，以便使用 Unity 的 AR Foundation 進行開發。

確認裝置是否支援 AR

Android 裝置上的 AR 體驗是由 ARCore 驅動，這項技術適用於[支援 ARCore 的裝置](#)。確認開發裝置與 AR 相容。或者，您也可以使用正確設定的[支援 AR 的 Android 模擬器例項](#)。

在裝置上設定 USB 偵錯

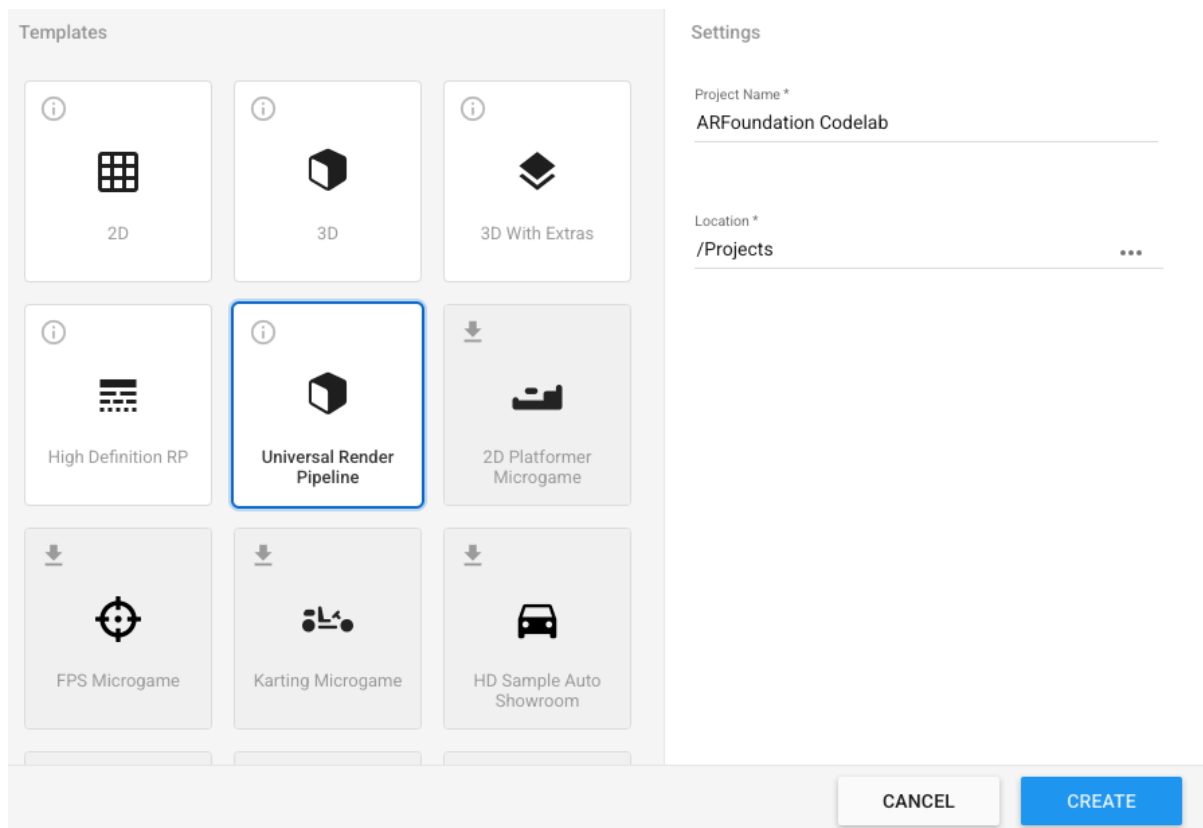
您必須在裝置上啟用「開發人員選項」，才能執行偵錯應用程式。如果尚未啟用，請參閱 Android 說明文件，瞭解如何[啟用開發人員選項](#)和 [USB 偵錯](#)。

安裝 Unity (2020.3 LTS)

在工作站上安裝 [Unity 2020 LTS](#)。在本程式碼研究室中，螢幕截圖顯示的是 **2020.3 (LTS)** 版本的 Unity UI。其他 Unity 版本可能也適用，但可能需要執行額外步驟。實際畫面可能與這裡顯示的螢幕截圖不同。

建立新專案

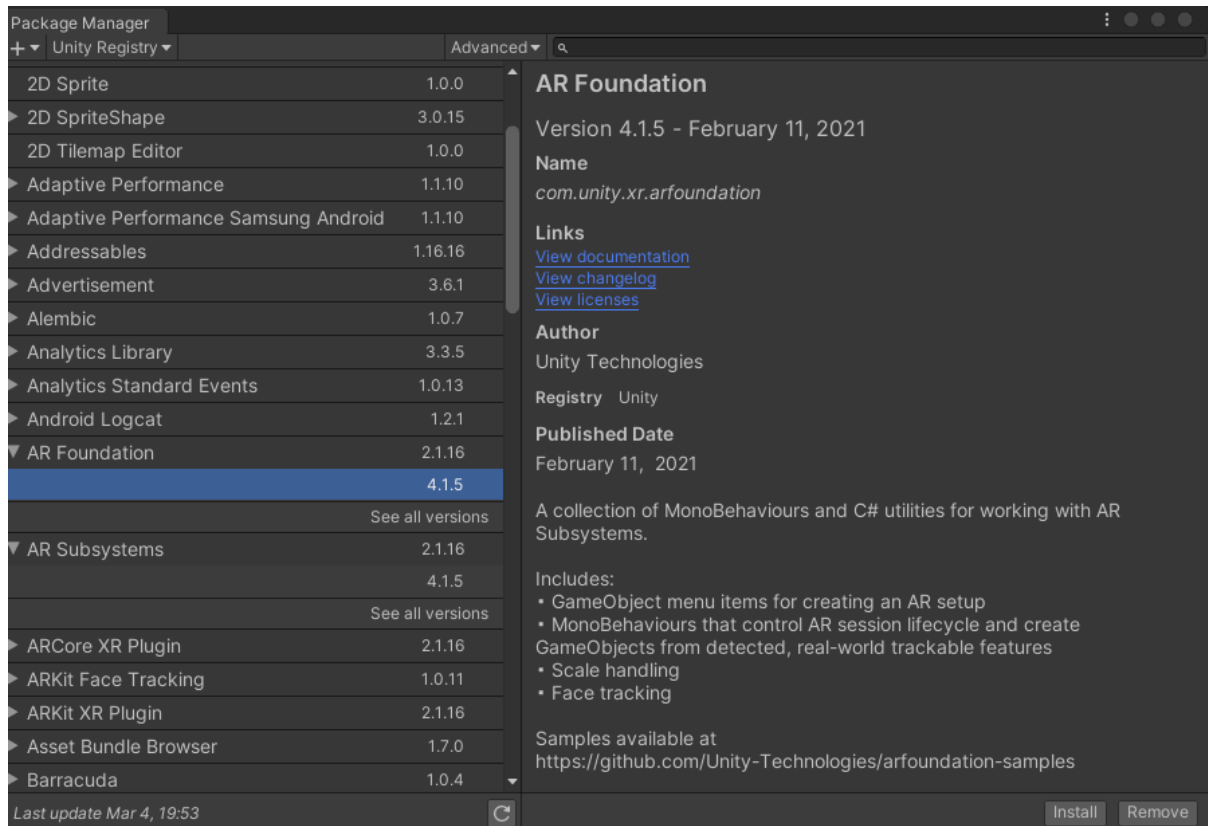
使用「通用算繪管道」範本建立新專案。為專案命名並選擇適當的位置，然後按下「建立」。



安裝必要架構

Unity 的 AR Foundation 位於 Unity Package Manager 中。

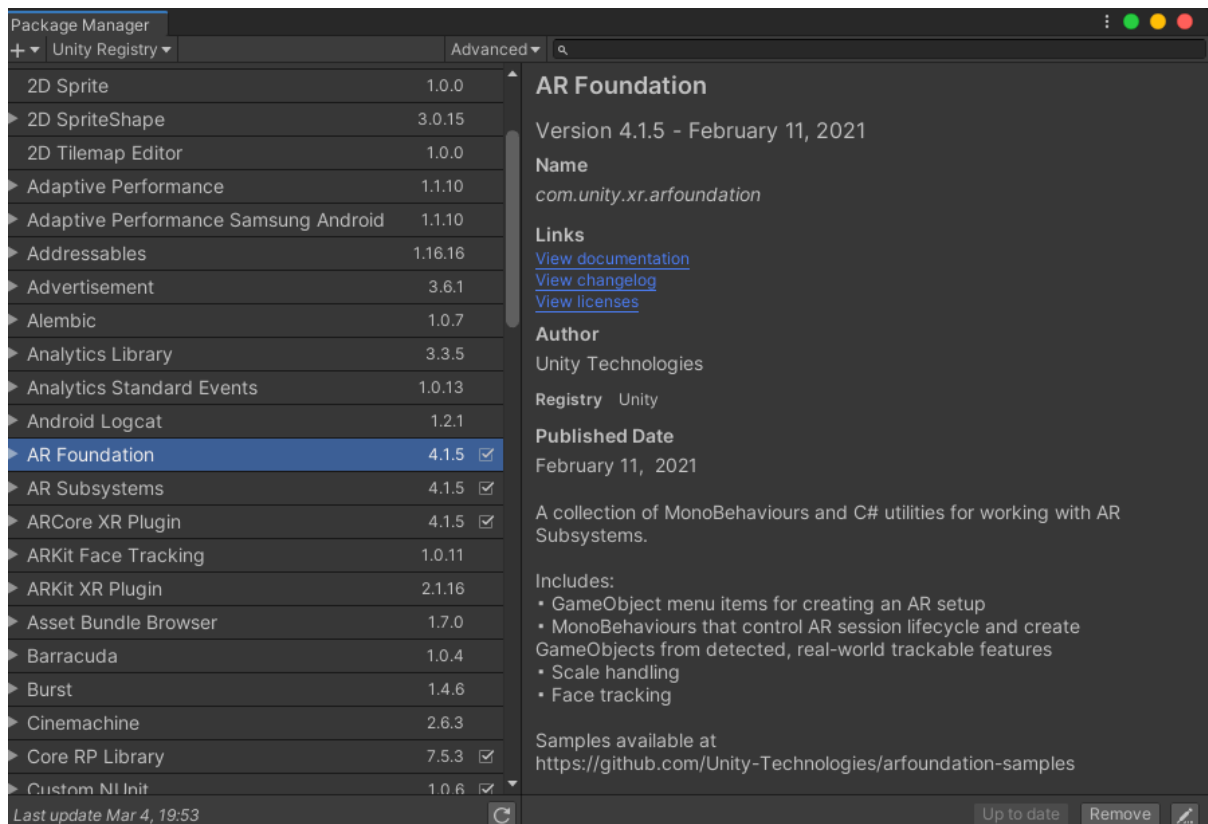
1. 依序點選「Window」>「Package Manager」，開啟套件管理工具。



2. 在這個視窗中，安裝本程式碼研究室中會用到的套件。使用 ► 圖示展開項目，即可查看這些架構的最新版本。為下列架構安裝最新版本：

- AR Foundation
- ARCore XR 外掛程式

完成後，套件管理員應如下所示：



安裝入門套件

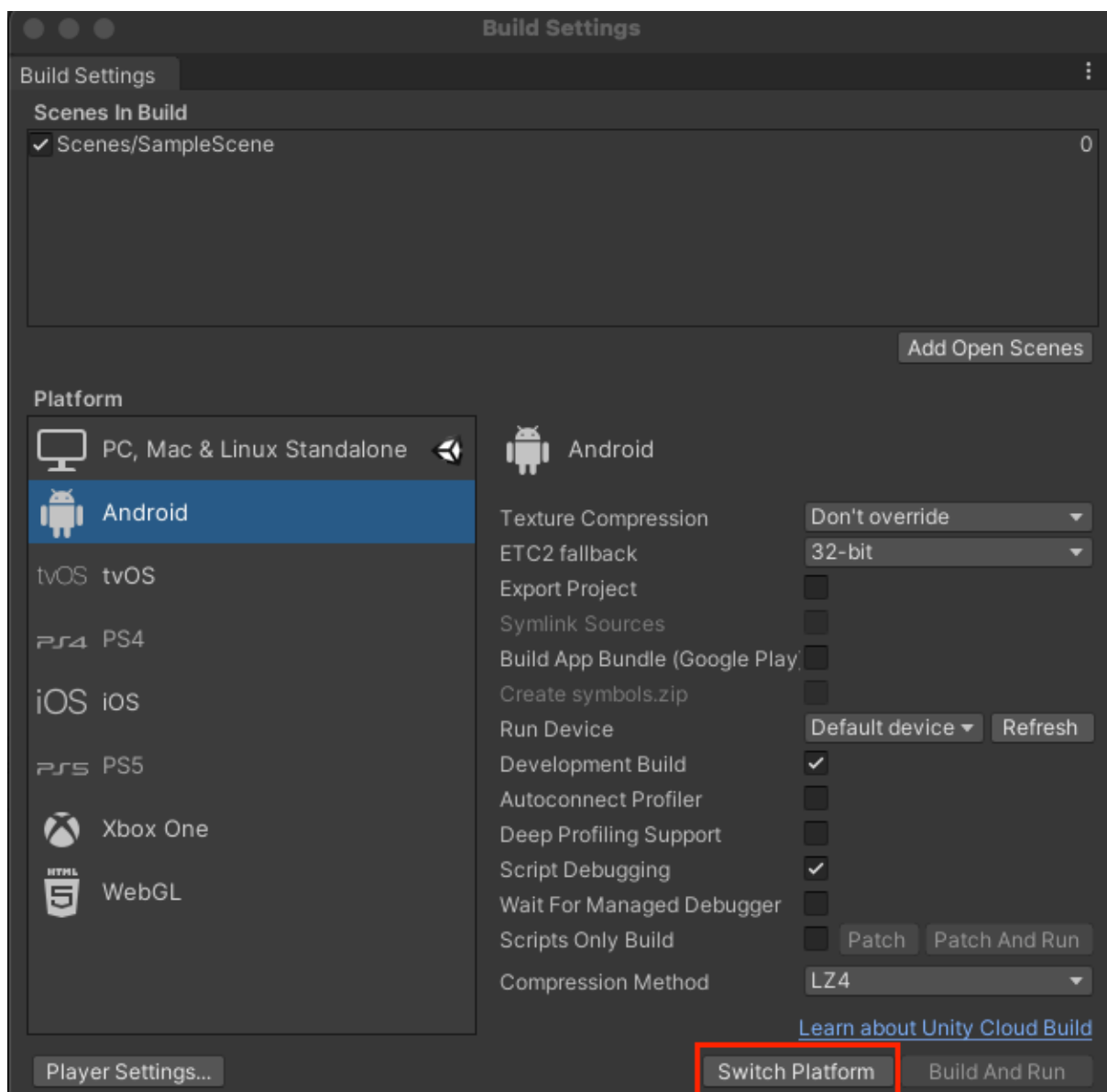
在本程式碼研究室中，我們提供了一個入門套件，內含預先製作的物件和指令碼，可加快部分程式碼研究室的進度，讓您專注於瞭解如何使用 AR Foundation。

- 1.
2. 開啟「Assets」>「Import Package」>「Custom Package...」，然後開啟 `starter-package.unitypackage`，即可安裝入門套件。
3. 在彈出式視窗中，確認已選取所有項目。
4. 按一下「匯入」。

變更建構設定

由於應用程式會在 Android 上執行，請將建構平台變更為 Android：

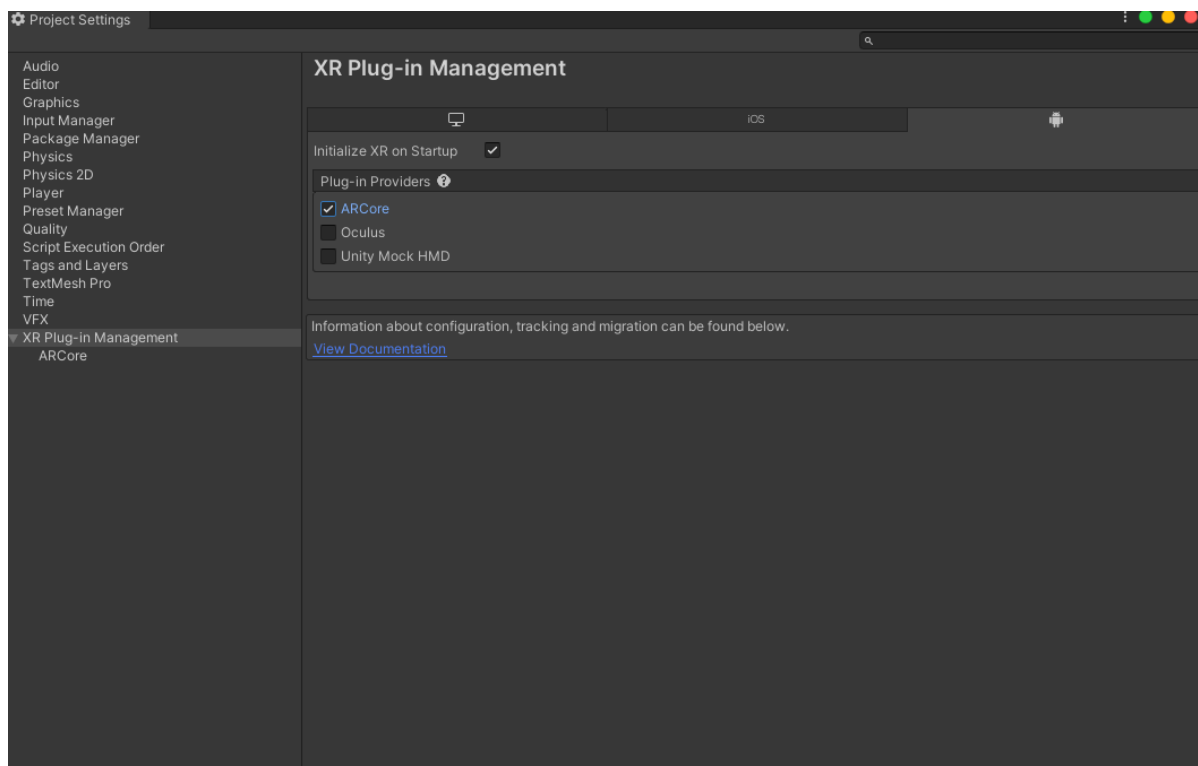
1. 依序開啟「File」>「Build Settings」。
2. 在「Platform」窗格中，選取「Android」。
3. 您可以視需要啟用「開發版本」和「指令碼偵錯」，以便在應用程式執行時保留偵錯資訊。
4. 按一下「切換平台」。



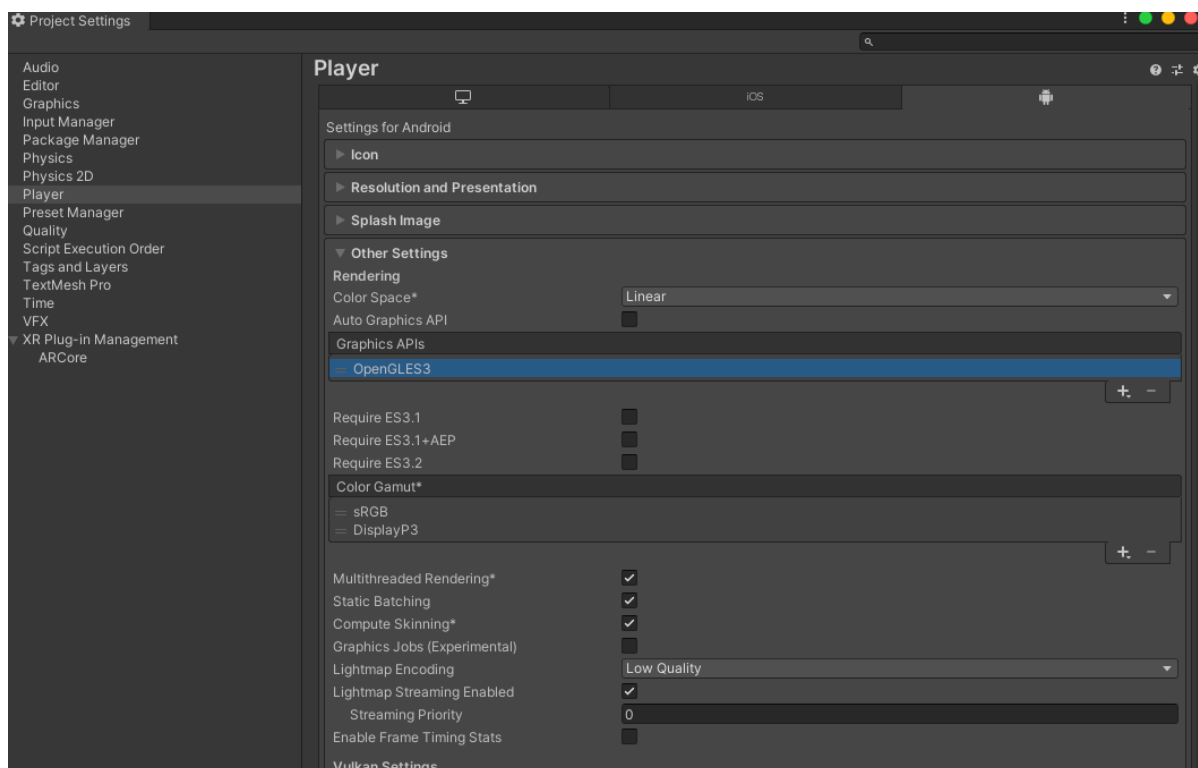
變更專案設定

AR Foundation 必須經過設定，才能在啟動時初始化 XR 系統。

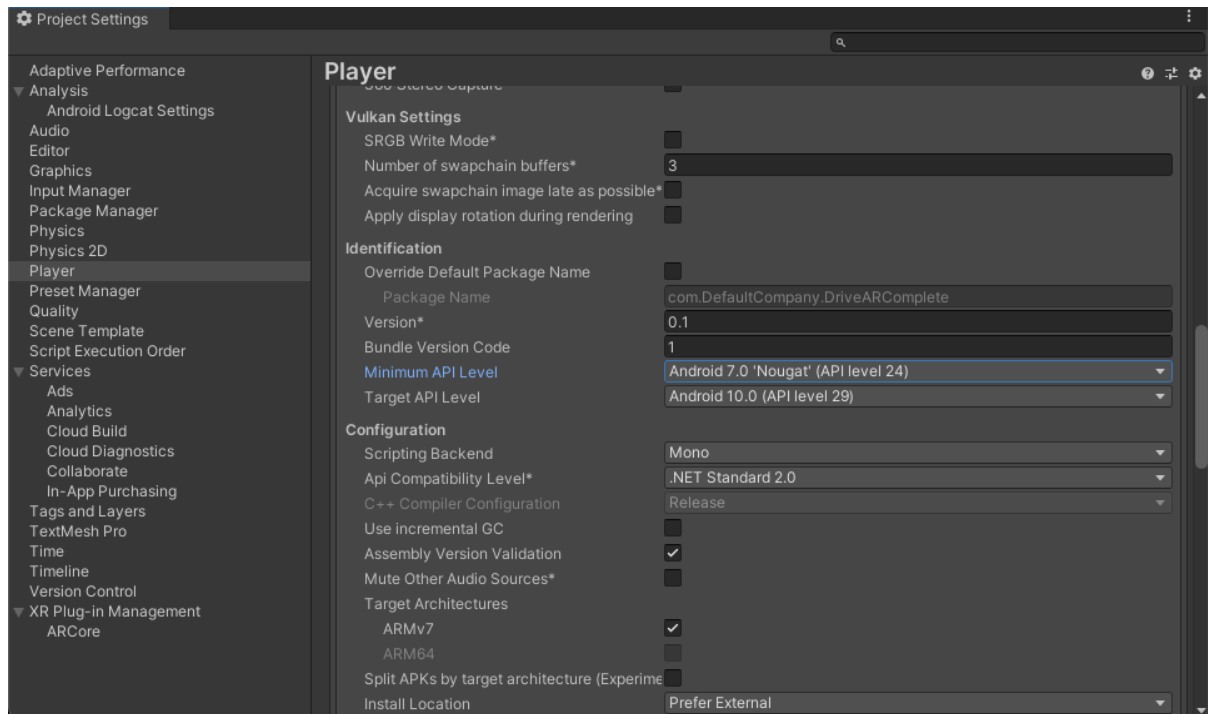
1. 開啟「Edit」>「Project Settings...」，然後按一下「XR Plug-in Management」部分。
2. 在「Android」分頁中，啟用「ARCore」。



3. 按一下左側窗格中的「播放器」部分。
4. 在「Android」分頁的「Other Settings」下方，從「Graphics APIs」移除「Vulkan」。



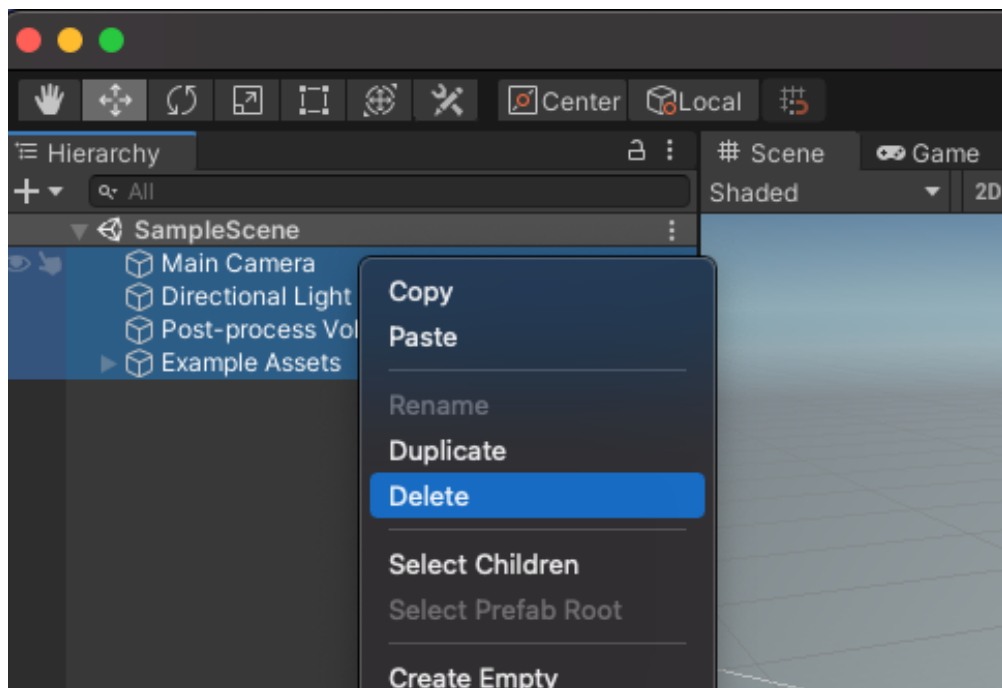
5. 使用 ARCore 的「必須支援 AR」應用程式需要至少 API 級別 24。向下捲動並找出「最低 API 級別」。將最低 API 級別設為 24。



新增必要的場景元素

通用轉譯管道範本隨附一些遊戲物件，但您在本教學課程中不會用到這些物件。

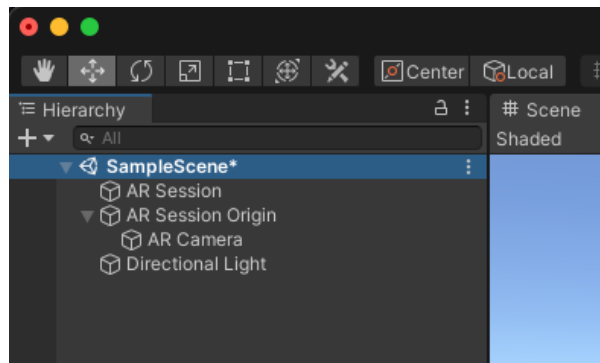
1. 刪除 `SampleScene` 中的所有遊戲物件。



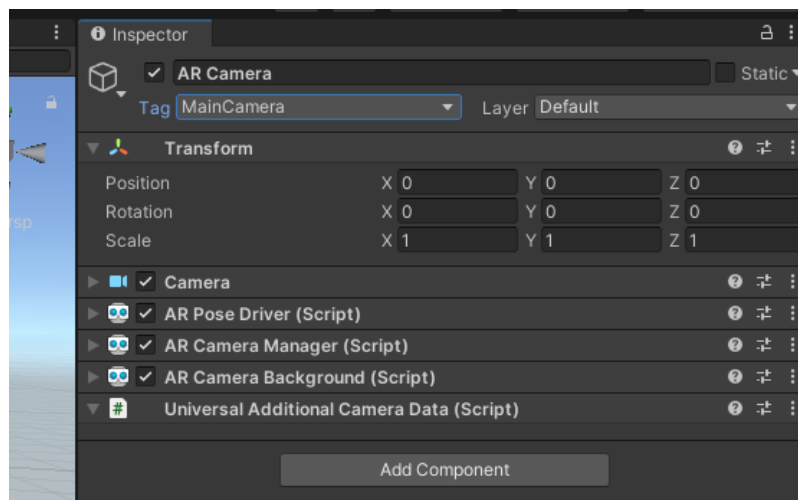
2. 新增 AR Foundation 物件。在「Hierarchy」(階層) 窗格中按一下滑鼠右鍵。使用這個選單可新增：

- **XR > AR Session**：這個物件可控管 AR 體驗的生命週期。
- **XR > AR Session Origin**：這個物件會將 AR 座標轉換為 Unity 世界座標。
- **Light > Directional Light**：提供光源，照亮遊戲物件。

階層應如下所示：



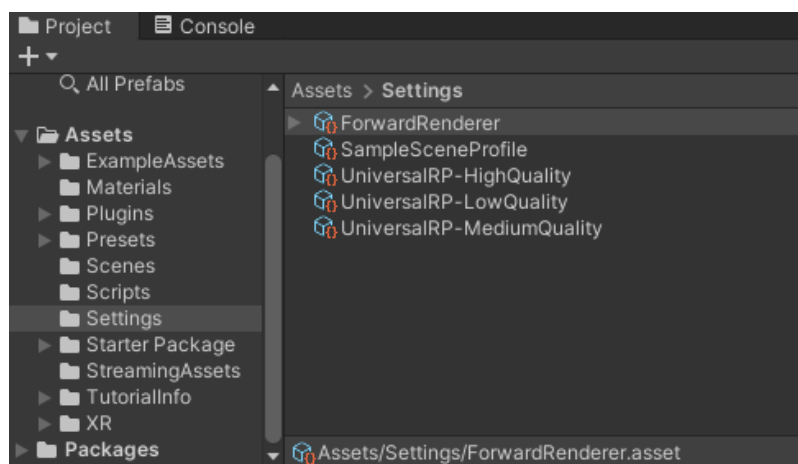
3. 展開您在階層中建立的「AR Session Origin」，然後選取「AR Camera」物件。在檢查器中，將標記變更為 **MainCamera**。



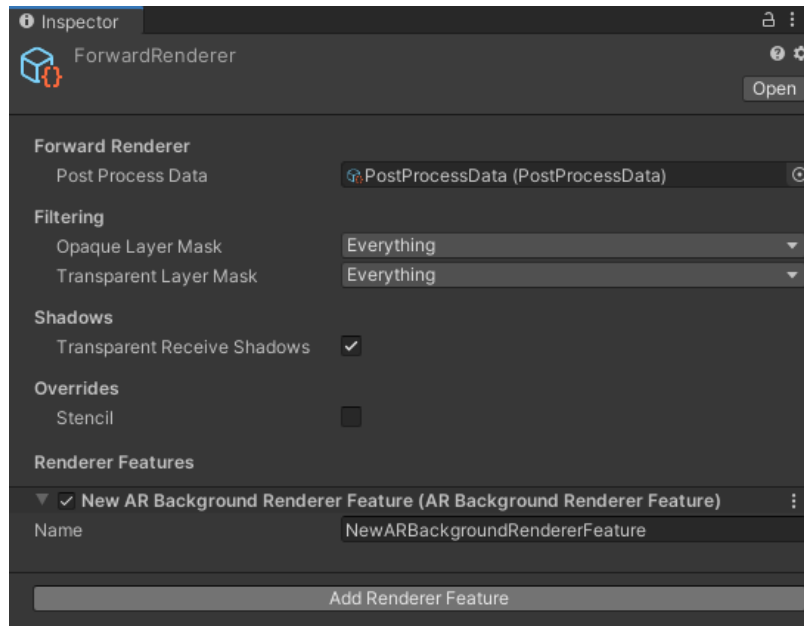
設定算繪

Unity 的通用算繪管線需要進行一項變更，才能與 AR Foundation 相容。

1. 在「Project」窗格中，依序前往「Assets」>「Settings」，找出「ForwardRenderer」資產。



2. 選取「ForwardRenderer」 **ForwardRenderer**。
3. 在「檢查器」窗格中，使用「Add Renderer Feature」新增「AR Background Renderer Feature」。這個元件會在場景中算繪攝影機動態饋給。



驗證設定

1. 確認裝置已接上電源，且已開啟 ADB 偵錯功能。
2. 依序點選「File」>「Build And Run...」這會將應用程式上傳至裝置，並在安裝完成後啟動。
3. 裝置螢幕上應會顯示攝影機畫面。

在下一個步驟中，您將開始為應用程式新增功能。

步驟 3 · 約 0 分鐘

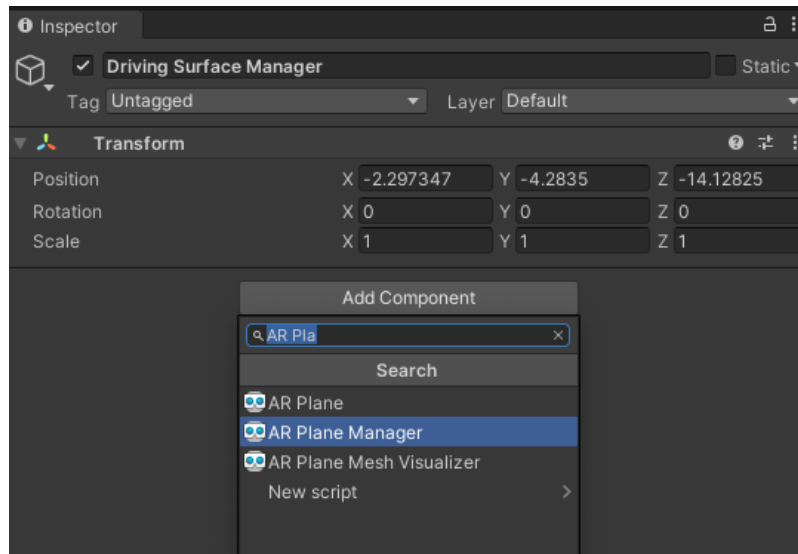
3. 偵測現實世界中的飛機

基本場景設定完成後，即可開始開發遊戲。在這個步驟中，您將偵測平面並繪製到場景中。

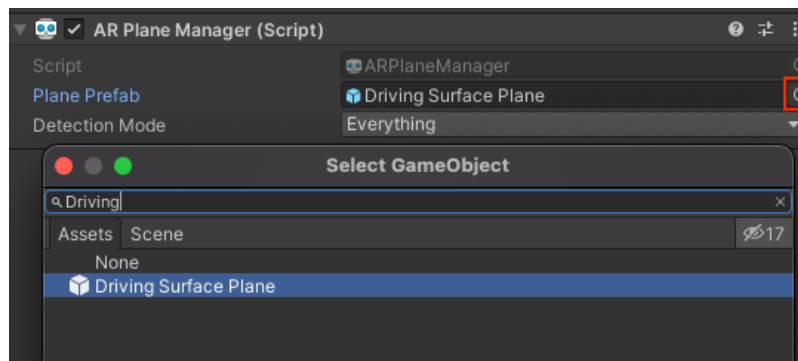
新增 **ARPlaneManager** 元件

ARPlaneManager 會偵測 **ARPlane**，並在裝置對環境的瞭解程度改變時，建立、更新及移除遊戲物件。

1. 使用「Hierarchy」窗格建立空白的 **GameObject**。
2. 將其重新命名為 **Driving Surface Manager**。這個元件會顯示飛機，直到玩家選取其中一架為止。
3. 選取新的遊戲物件。在「檢查器」窗格中，按一下「新增元件」，新增「AR 平面管理工具」。



4. 設定 **Plane Prefab** 欄位來設定 **ARPlaneManager**：
 1. 按一下 **None** 旁的按鈕，開啟「Select GameObject」(選取 **GameObject**) 視窗。
 2. 選取「資產」分頁標籤，然後搜尋「Driving Surface Plane」。



這個預先建構的物件來自入門套件，提供粗糙的地板紋理，將做為平面裝飾。

5. 將 **Detection Mode** 變更為 **Horizontal**。這會將 **ARPlaneManager** 設為只提供水平平面，非常適合在上面行駛。

新增 **ARRaycastManager** 元件

ARRaycastManager 會公開光線投射功能。在下一個步驟中，我們會使用這個物件為使用者提供控制項。

1. 確認已在「Hierarchy」窗格中選取名為 **Driving Surface Manager** 的物件。

2. 在「檢視器」中，按一下「新增元件」，將 `ARRaycastManager` 元件新增至遊戲物件。

這個元件不需要進一步設定。

新增 `DrivingSurfaceManager` 元件

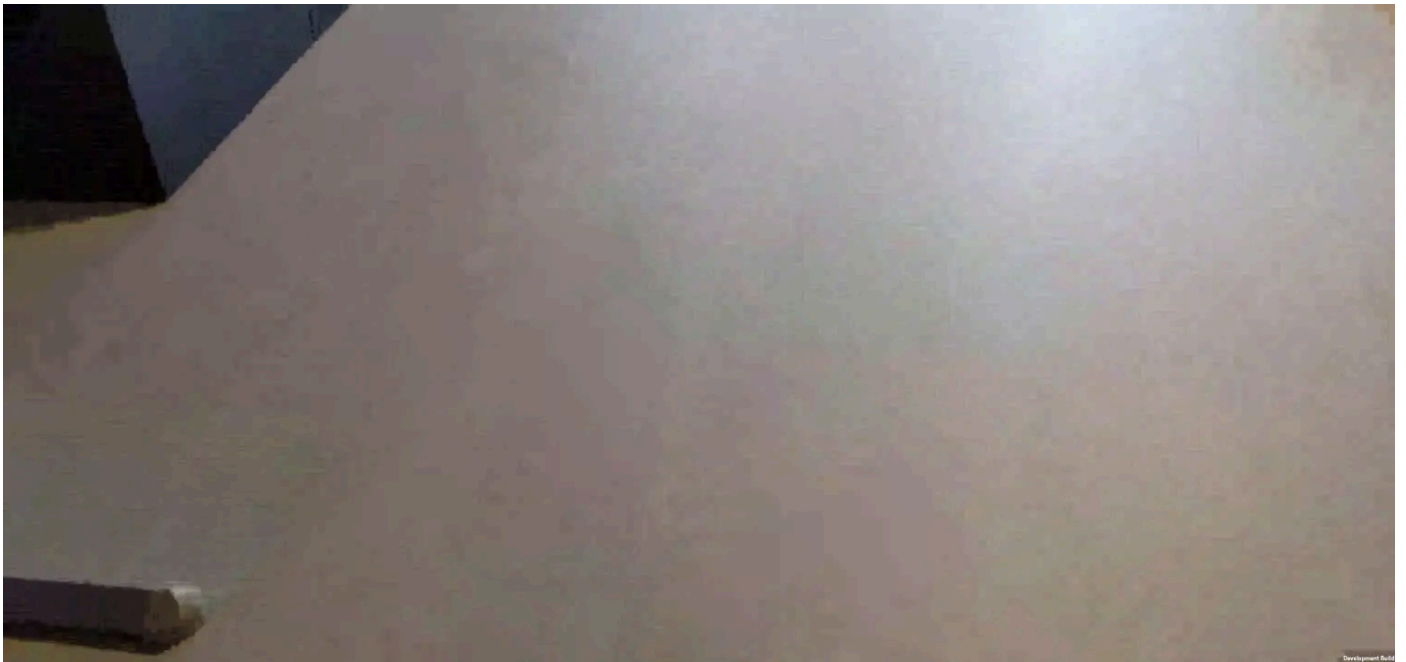
`DrivingSurfaceManager` 是 Starter Package 中的輔助指令碼，可選取 `ARPlane`。選取 `ARPlane` 後，系統會隱藏所有其他平面，並停用新平面。

1. 確認已在「Hierarchy」窗格中選取名為 `Driving Surface Manager` 的物件。
2. 在「檢查器」中，按一下「新增元件」，將 `DrivingSurfaceManager` 元件新增至遊戲物件。

這個元件不需要進一步設定。

執行應用程式

1. 按一下「File」>「Build And Run...」即可測試變更。
2. 將裝置對準現實世界中的水平表面，並移動裝置，讓 ARCore 更瞭解周遭環境。



3. ARCore 偵測到平面後，您應該會看到覆蓋在實體表面上的泥土紋理。`ARPlaneManager` 會為每個偵測到的飛機例項指定的 `Plane Prefab`。`Driving Surface Plane` 預先建構元件具有 `ARPlaneMeshVisualizer` 元件，可為指定 `ARPlane` 建立網格。

在下一個步驟中，您將使用偵測到的平面做為遊戲場地。

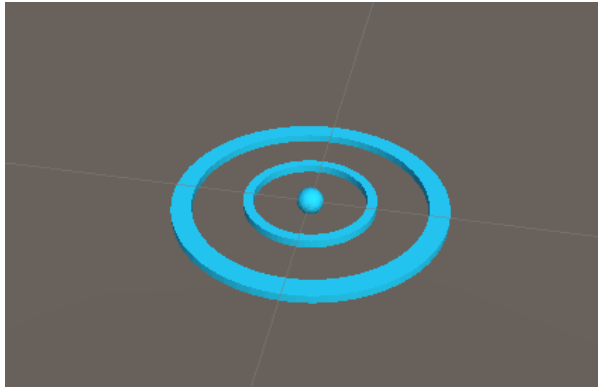
步驟 4 · 約 0 分鐘

4. 對偵測到的平面執行命中測試

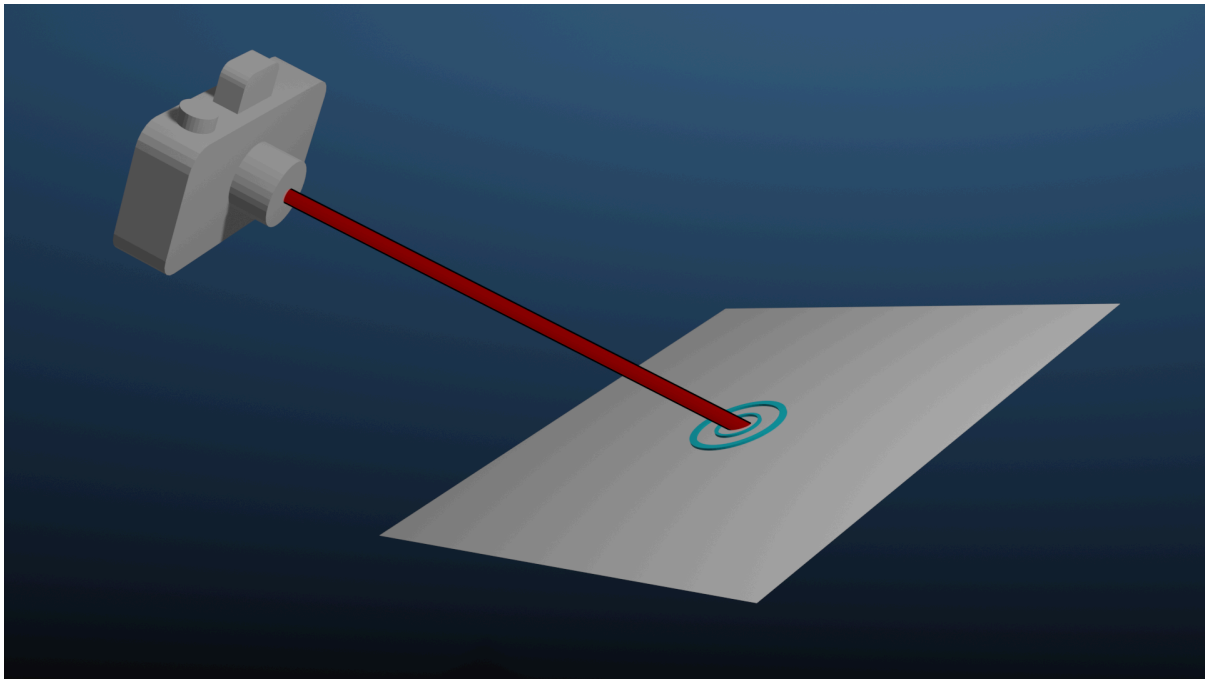
在上一步中，您已編寫可偵測平面圖的應用程式。這些平面會反映在遊戲場景中。現在，您要建立瞄準十字線和車輛，在偵測到的平面上行駛，藉此新增與這些平面的互動。

建立瞄準線

這個應用程式的控制方式是讓玩家將手機對準某個表面。如要針對指定位置提供清楚的視覺意見回饋，請使用瞄準十字線。



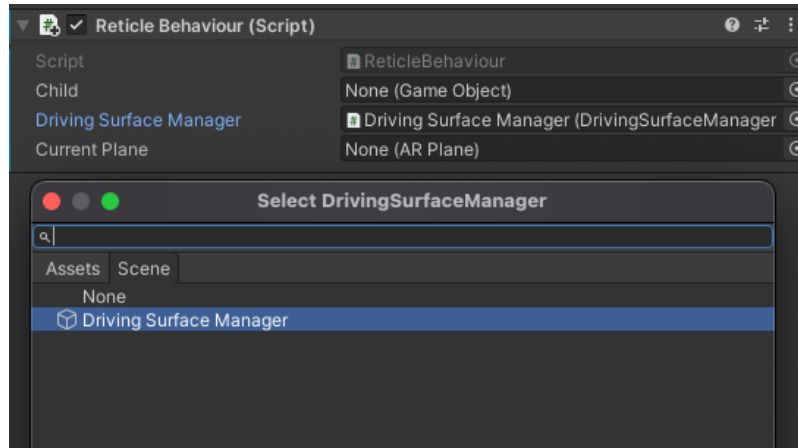
如要將十字線「黏貼」到 AR 平面，請使用命中測試。命中測試是一種技術，可在特定方向投射光線時計算交集。您將使用命中測試，偵測攝影機視野方向的交集。



新增十字線

1. 在畫面底部的「Project」窗格中，依序前往「Assets」>「Starter Package」。
2. 將「Reticle Prefab」拖曳至專案的「Hierarchy」窗格，將其放入場景中。
3. 在階層中選取十字線。
4. 在檢查器中，按一下「Add Component」(新增元件)。從 Starter Package 新增 `ReticleBehaviour` 指令碼。這個指令碼包含一些用於控制十字線的樣板。

5. `ReticleBehaviour` 指令碼取決於您先前建立的 `Driving Surface Manager`，因此請點選 `Driving Surface Manager` 選擇器，新增依附元件。選取「場景」分頁標籤，然後選擇 `Driving Surface Manager`。



編輯「`ReticleBehaviour`」

`ReticleBehavior` 指令碼會將瞄準線放置在裝置可視區域中央的平面上。

1. 按兩下 `Script` 欄位，開啟 `ReticleBehaviour.cs` 指令碼。
2. 使用「相機」`ViewToScreenPoint` 判斷螢幕中心。編輯 `Update()` 方法，加入下列內容：

```
var screenCenter = Camera.main.ViewportToScreenPoint(new Vector3(0.5f, 0.5f));
```

3. 使用這個點執行光線投射。新增下列程式碼：

```
var hits = new List<ARRaycastHit>();  
DrivingSurfaceManager.RaycastManager.Raycast(screenCenter, hits,  
TrackableType.PlaneWithinBounds);
```

如果在 `Camera.main` 中遇到 `NullReferenceException`，請確認 **Hierarchy > AR Session Origin** 中的場景 `AR Camera` 已標記為 **MainCamera**。

變數 `hits` 會包含 `ARRaycastHit`，說明 `ray` 相交的可追蹤項目上的點。

`TrackableType.PlaneWithinBounds` 是 `TrackableType` 中的值。這項設定會指定可透過光線投射偵測到的 `Trackable` 種類。詳情請參閱 Unity 的 [TrackableType](#) 說明文件。

4. 查詢 `hits` 清單，找出感興趣的交集點。優先使用 `DrivingSurfaceManager` 中包含的鎖定平面，如果不存在，則使用第一個平面命中。在 `Update()` 結尾新增下列內容：

```

CurrentPlane = null;
ARRAYcastHit? hit = null;
if (hits.Length > 0)
{
    // If you don't have a locked plane already...
    var lockedPlane = DrivingSurfaceManager.LockedPlane;
    hit = lockedPlane == null
        // ... use the first hit in `hits`.
        ? hits[0]
        // Otherwise use the locked plane, if it's there.
        : hits.SingleOrDefault(x => x.trackableId == lockedPlane.trackableId);
}

```

5. 如果 `hit` 包含結果，請將這個 `GameObject` 的轉換移至命中位置。

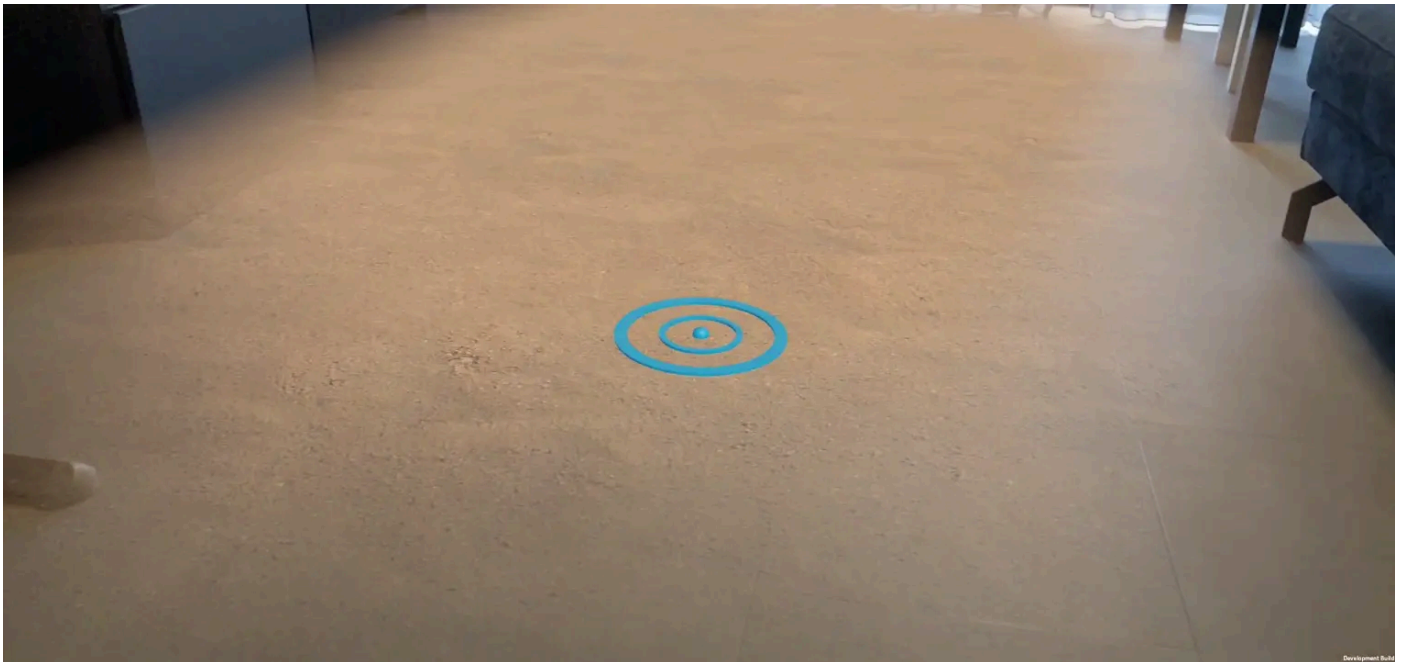
```

if (hit.HasValue)
{
    CurrentPlane = DrivingSurfaceManager.PlaneManager.GetPlane(hit.Value.trackableId);
    // Move this reticle to the location of the hit.
    transform.position = hit.Value.pose.position;
}
Child.SetActive(CurrentPlane != null);

```

測試十字線

1. 按一下「File」>「Build And Run...」即可測試變更。
2. 將裝置對準飛機時，十字線應會跟著攝影機移動。



建立車輛

玩家將控制玩具車，朝著瞄準線的位置行駛。入門套件中提供這輛車的模型和行為。

在場景中新增 `CarManager`

1. 在「Hierarchy」中，建立新的空白 `GameObject`。
2. 將其重新命名為 `Car Spawner`。
3. 選取您建立的物件。在「Hierarchy」(階層) 窗格中，按一下「Add Component」(新增元件)，新增 `CarManager` 元件。
4. 點選每個欄位的選擇器，設定 `CarManager` 的依附元件：
 - **車輛 Prefab**：在「Assets」中選取「Car Prefab」。
 - **十字線**：在「場景」中，選取「十字線預製物件」。
 - **行車路面管理工具**：在「場景」中，選取「行車路面管理工具」。

這項 `CarManager` 行為會在十字線所在的平面上生成玩具車。如要瞭解車輛的程式設計方式，請查看 `CarBehaviour` 指令碼。

試駕

1. 按一下「File」>「Build And Run」，測試變更。
2. 輕觸飛機後，該位置應該會顯示一輛小車。這輛車會跟隨十字線。

新增遊戲元素

現在玩家可以控制場景中的實體，請為玩家提供行車目的地。

1. 在「Hierarchy」中建立新的空白 `GameObject`。
2. 將其重新命名為 `Package Spawner`。
3. 選取您建立的物件。在「Hierarchy」(階層) 窗格中，按一下「Add Component」(新增元件)，將 `PackageSpawner` 元件新增至其中。
4. 點選每個欄位的選擇器，設定 `PackageSpawner` 的依附元件：
 - **套件 Prefab**：在「Assets」(資產) 中，選取「Package Prefab」(套件 Prefab)。
 - **駕駛路面管理工具** 在「Scene」中，選取「Driving Surface Manager」。

如果沒有包裹，這個 `PackageSpawner` 行為會在鎖定的 `ARPlane` 上隨機位置產生新包裹。

測試遊戲

1. 按一下「File」>「Build And Run」，測試變更。
2. 建立車輛後，系統應會產生套件。
3. 開車前往包裹所在位置。
3. 新的圖示會隨機出現在畫面上。

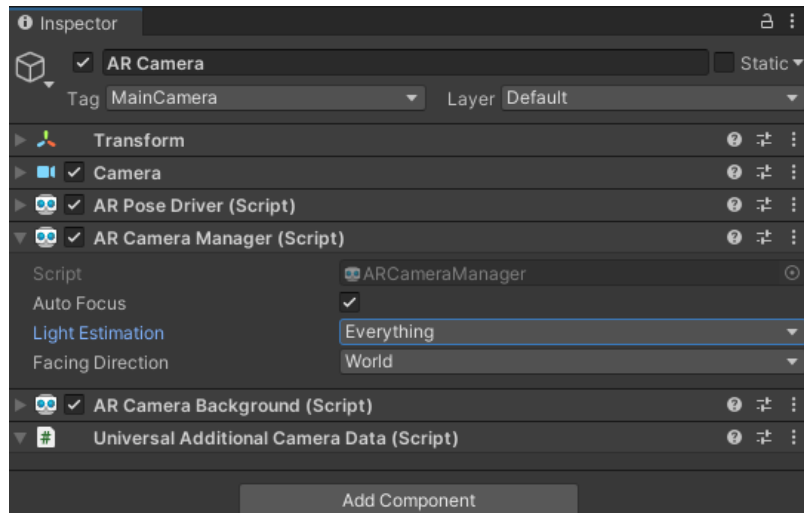


5. 設定光線估算

基本遊戲已完成，現在為擴增實境場景增添真實感。在本步驟中，您將使用 ARCore 的 Lighting Estimation API，根據傳入的攝影機畫面偵測現實世界中的光照。這項資訊會用於調整場景的光照，使其符合現實世界的光照。

啟用光線估算功能

1. 在「Hierarchy」中，展開「AR Session Origin」，然後選取「AR Camera」物件。
2. 在「檢查器」中，展開「AR Camera Manager」指令碼。
3. 將「光線估算」欄位變更為「所有內容」。



修改方向光

1. 在「Hierarchy」(階層) 中，選取「Directional Light」(方向光) 物件。
2. 將 `LightEstimation` 元件新增至其中。這個 Starter Package 的元件提供一些樣板，用於訂閱光照變化。
3. 在 `FrameReceived()` 函式中新增：

```

ARLightEstimationData lightEstimation = args.lightEstimation;

if (lightEstimation.averageBrightness.HasValue)
    Light.intensity = lightEstimation.averageBrightness.Value;

if (lightEstimation.averageColorTemperature.HasValue)
    Light.colorTemperature = lightEstimation.averageColorTemperature.Value;

if (lightEstimation.colorCorrection.HasValue)
    Light.color = lightEstimation.colorCorrection.Value;

if (lightEstimation.mainLightDirection.HasValue)
    Light.transform.rotation =
Quaternion.LookRotation(lightEstimation.mainLightDirection.Value);

if (lightEstimation.mainLightColor.HasValue)
    Light.color = lightEstimation.mainLightColor.Value;

if (lightEstimation.mainLightIntensityLumens.HasValue)
    Light.intensity = lightEstimation.averageMainLightBrightness.Value;

if (lightEstimation.ambientSphericalHarmonics.HasValue)
{
    RenderSettings.ambientMode = AmbientMode.Skybox;
    RenderSettings.ambientProbe = lightEstimation.ambientSphericalHarmonics.Value;
}

```

如要進一步瞭解各項參數，請參閱 [AR Foundation 說明文件](#)。 `ARLightEstimationData`

測試變更

1. 按一下「File」>「Build And Run」，測試變更。
2. 查看場景中的物體時，您可能會發現物體會根據環境的光線著色。
3. 請盡量調整燈光。舉例來說，請嘗試關閉所在房間的燈。您應該會看到物體上的光線會配合現實世界的光線變化調整。



6. 總結

恭喜！您已完成本 Unity AR Foundation 程式碼研究室。

涵蓋內容

- 如何使用 Unity 的 AR Foundation 和通用算繪管道設定基本專案。
- 如何使用 `ARPlaneManager` 訂閱新方案。
- 如何使用 `Raycast` 尋找與虛擬幾何圖形的交集。
- 如何使用 `ARLightEstimationData` 照亮場景。

後續步驟

- 請參閱 [Unity 的 AR Foundation 範例](#)。
- 瀏覽 [AR Foundation 說明文件](#)。
- 請參閱 [Google 說明文件](#)，瞭解 Unity 的 AR Foundation 適用的 ARCore 擴充功能。

額外作業

如要擴充您在這裡建立的遊戲，可以考慮下列想法：

- 在 `PackageManager` 產生新套件時修改 `TextMeshPro`，即可在遊戲中新增分數計數器。
 - 啟用「效能疊加」，即可在遊戲執行時查看效能資訊。
 - 請先使用[持續性光線投射](#)在場景中放置新物件。偵測到該區域中的平面時，該物件會更新並貼齊平面。
-